Text Vectorization

- Representing textual information in numerical format that can be effectively utilized by computational models
- Once text is vectorized, all language problems become spatial geometry problems.
- Distance or angle between points in a multi-dimensional space are used to match, group, or organize documents

NLP Applications

- Semantic Search & Retrieval
- Document Classification & Clustering
- Document Summarization
- Plagiarism Detection

1. Conceptually related content	matching query vector and a document vector
2. Grouping or sorting similar articles	grouping document vectors
3. Condensing large bodies of text	matching sentence vector and the document vector
4. Spotting paraphrased or near-duplicate texts	matching sentence/document vectors

Similarity Measures

- **Cosine Similarity:** Measures the *direction* the vectors are pointing
- **Euclidean Distance:** Measures the physical distance between two points. Highly effective for fixed-length embeddings and clustering
- **Dot Product:** Frequently used by high-scale vector databases

Text Vectorization Approaches

1. Statistical/Count-Based Methods

- **Bag-of-Words (BoW):** represents a document as a vector where each dimension corresponds to a unique word in the entire corpus's vocabulary, and the value represents the frequency of that word in the document.
- It captures word frequency but disregards the order and grammatical structure of words.
- Frequency quantifies term relevance; used to compute similarity and document ranking.
- **Term Frequency-Inverse Document Frequency (TF-IDF):** assigns weights to words based on how frequently they appear in a specific document **relative to their frequency across the entire collection of documents.** This helps highlight words that are more informative and specific to a particular document, giving less weight to common words like "the" or "a".
- **One-Hot Encoding:** represents words by a sparse binary vector of the size of the vocabulary, with a "1" at the index corresponding to the word and "0" elsewhere. This is simple but computationally inefficient for large vocabularies and doesn't capture semantic relationships

Bag of Words Model explanation

Corpus

A collection of text documents

↓

Tokenize

Divide the text into smaller units called tokens, usually words or phrases

↓

Count word frequencies

Create a vocabulary of unique words and count the number of times each word appears in each document.

↓

Encode the data

Encoding the text data as numerical values by creating a vector for each document, with each element of the vector representing the frequency count of a particular word in the document

© AIML.com Research

Example

Corpus:

The dog is happy. The child makes the dog happy. The dog makes the child happy

↓

Tokenization:

D1: [The] [dog] [is] [happy]
D2: [The] [child] [makes] [the] [dog] [happy]
D3: [The] [dog] [makes] [the] [child] [happy]

↓

Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1

↓

Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	**[0,1,1,1,0,1]**
D2	1	1	1	0	1	2	**[1,1,1,0,1,2]**
D3	1	1	1	0	1	2	**[1,1,1,0,1,2]**

Vectorization using TF-IDF

- TF-IDF turns each sentence (or document) into a vector based on count and proportions.
- **TF (Term Frequency):** how often a word appears in a sentence.
- **IDF (Inverse Document Frequency):** how rare that word is across all sentences or documents.

$$\mathrm{IDF}(t) = \log\left(\frac{N}{\mathrm{DF}(t)}\right)$$

where
N= total number of sentences,
DF(t) =number of sentences containing the word t.

Combined weight

$$\mathrm{TF{-}IDF}(t,d) = TF(t,d) \times IDF(t)$$

- Sentence 1 : The car is driven on the road.
- Sentence 2: The truck is driven on the highway.

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	1/7	1/7	log(2/2) = 0	0	0
Car	1/7	0	log(2/1) = 0.3	0.043	0
Truck	0	1/7	log(2/1) = 0.3	0	0.043
Is	1/7	1/7	log(2/2) = 0	0	0
Driven	1/7	1/7	log(2/2) = 0	0	0
On	1/7	1/7	log(2/2) = 0	0	0
The	1/7	1/7	log(2/2) = 0	0	0
Road	1/7	0	log(2/1) = 0.3	0.043	0
Highway	0	1/7	log(2/1) = 0.3	0	0.043

Cosine Similarity Measure

Cosine similarity measures how similar two vectors are by checking the angle between them.

- If two vectors point in **exactly the same direction**, similarity = **1**
- If they are **orthogonal** (90° apart), similarity = **0**
- If they point in **opposite directions**, similarity = **−1**

	Word2Vec	BERT
Word sense Interpretation	Word vector ↔ average of nearby word vectors	Target token embedding ↔ context/sense embeddings
	Closest cluster indicates sense	Closest contextual/synset match indicates sense

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Where:

- $A \cdot B$ = **dot product** of vectors A and B

$$A \cdot B = \sum_{i=1}^{n} (A_i \times B_i)$$

- $\|A\|$ = **magnitude** (length) of vector A

$$\|A\| = \sqrt{\sum_{i=1}^{n} A_i^2}$$

- $\|B\|$ = **magnitude** of vector B

$$\|B\| = \sqrt{\sum_{i=1}^{n} B_i^2}$$

TF-IDF Vectors and cosine similarity

- **Sentence A:** *"A semaphore is an integer variable used to solve critical section problems by achieving process synchronization."*
- **Sentence B:** *"A semaphore is a numeric variable utilized to handle critical section issues by providing process synchronization."*

$$IDF = \ln(N/DF) + 1$$

Vector weights: `[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.693, 1.693, 1.693, 1.693, 1.693, 0, 0, 0, 0, 0]`

Vector weights: `[1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 0, 0, 0, 0, 0, 1.693, 1.693, 1.693, 1.693, 1.693]`

$$\text{Similarity} = 6.0/(4.51 \times 4.51) = \mathbf{0.295}$$

Semantic Similarity: Word Embeddings

2. Learning-Based/Dense Vector Methods

- **Word Embeddings:** Words mapped to a continuous vector space where the distance and direction between vectors reflect the semantic similarity and relationships between the corresponding words.
- **Word2Vec:** Transform words into dense, lower-dimensional vectors (embeddings) using neural networks
- Learns word associations from a large corpus, generating highly correlated vector coordinates, for words frequently surrounded by similar set of words.
- In Word2Vec, every word in the vocabulary gets exactly **one vector** assigned to it.
- **Context-Aware Embeddings:** generate dynamic word representations that consider the surrounding words in a sentence
- **BERT (Bidirectional Encoder Representations from Transformers):** uses deep bidirectional neural network and a "masked language model" for enhanced contextual information

Word2Vec (Mikolov et al., 2013)

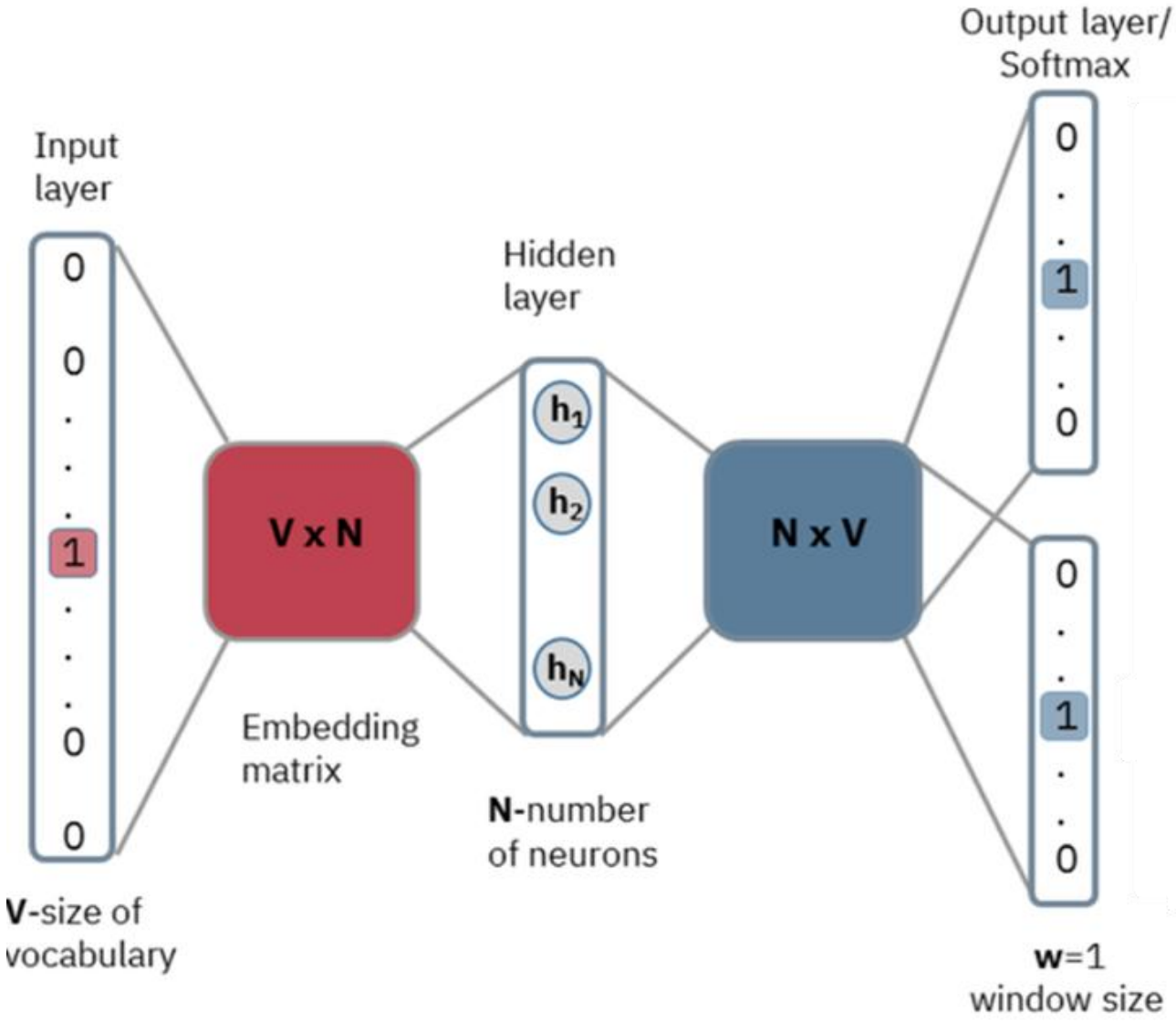
- Two architectures: Skip-Gram & CBOW
- uses Neural network with 01 hidden layer
  - **Input:** one-hot encoded word(s)
  - **Hidden layer:** linear layer with *no activation*
  - **Output:** probability distribution over the vocabulary (via softmax)
  - The weights learned between input and hidden (and hidden → output) become the word embeddings.

i. **Skip-Gram**
Predict *context words* given a *target word*.
Example (window = 2):
Sentence: “the little bird is eating”
For target “bird”, context = [the, little, is, eating]

ii. **CBOW (Continuous Bag of Words)**
Predict *target word* from its *context words*.
Example: Predict “bird” from [the, little, is, eating]

Word2Vec Skip-Gram architecture

- Trained on **each (target word, context word)** pair that appears within the defined **context window** in the corpus.
- Each such pair is treated as **one training sample** for the neural network.



Word2Vec CBOW architecture

- During training, the network learns two sets of vectors—one for context words and one for the target word—which are then combined (typically by averaging) to form a single vector for the context.
- The final learned vectors for each word are derived from the weights in the network's embedding layer after the training is complete.

